

Identifying Fraud Rings Through Clustering

IEEE-CIS Fraud Detection Dataset

Step 1: Problem Understanding & Framing

Business Problem

The IEEE-CIS Fraud Detection dataset comes from a partnership between the IEEE Computational Intelligence Society and Vesta Corporation, a leading payment services provider, built around a very ordinary business problem: a legitimate customer's card gets declined at checkout because a fraud model flagged it, creating friction, embarrassment, and lost sales; while a genuinely fraudulent transaction that slips through costs the platform directly. Vesta's stated goal for the Kaggle competition was to improve fraud-detection accuracy enough to address both failure modes at once: **(1) catching more real fraud** (increase True Positives) while **(2) cutting the false alarms** (decrease False Positives) that inconvenience honest customers and cost businesses revenue.

This is a problem that is close to home as e-commerce platforms lose significant revenue to fraud rings (a group of accounts, users, devices, etc. that work together to commit fraud). The whole Fraud Detection dataset mirrors the core challenge my team addressed in a **Fraud Clustering Project** deployed in 2025 by Shopee's PH BI & Fraud teams. Fraud agents were spending roughly 40% of their review time manually deciding whether a group of accounts (with similar delivery addresses, IP addresses, email domains, or recipient names) belonged to the same underlying fraud ring. Automating that grouping step was the highest-leverage intervention in the pipeline: correctly identifying one cluster shuts down every linked fraudulent account at once, instead of reviewing accounts one at a time.

Data Science Problem

Given anonymized e-commerce transaction and identity data, can transactions be grouped into clusters that correspond to the same underlying client, *without using the fraud label during clustering*, such that the resulting clusters concentrate known fraud well above the platform's base fraud rate?

Task Type

The project is under unsupervised learning, specifically clustering. The data provided by Kaggle has both training and test data, but only the training data includes the isFraud label; the official test set is unlabeled and used for Kaggle's own leaderboard scoring. Since validation depends on checking cluster fraud concentration against isFraud, I used a labeled hold-out split carved out of the training data for validation instead.

The objective of this project is to group transactions by similarity across identity and behavioral attributes (email domain, address/region proxies, device fingerprint, transaction timing), rather than classify each transaction individually. I withheld the isFraud label from the clustering fit and used it afterward as an external validation signal to check whether clusters concentrated known fraud. Because the dataset does not provide a ground-truth client or fraud-ring identifier, this analysis measures fraud concentration; it does not prove that each cluster represents one underlying client.

Note: The original Kaggle competition frames this as supervised binary classification (predict isFraud per transaction, scored on ROC-AUC). This capstone deliberately isolates the entity-clustering sub-problem instead, which, was also the key insight behind the competition's 1st-place "fraudsquad" solution (Yakovlev & Deotte, Kaggle Discussions, 2019)

Success Metrics

Technical: Elbow, Silhouette Score, Davies-Bouldin Index (cluster quality, no labels needed).

Business KPI:

- Confusion matrix results, treating the highest-risk cluster(s) as the predicted-fraud group, where a high % of True Positives (goal 1: catching more fraud) and a low % of False Positives (goal 2: cutting false alarms) confirm the clusters found are actually useful.

- Fraud-rate concentration: The share of *isFraud*=1 transactions captured within the highest-risk clusters, compared against the dataset's overall base fraud rate (~3.5%).
- Share of transactions that can be resolved as “likely same client” in bulk via cluster membership, rather than reviewed individually. This is the direct analogue to the ~40% agent-time savings measured in the real Shopee project.
- Reference point and not a target: The Kaggle competition's eventual 1st-place (fully supervised) solution reached a private leaderboard ROC-AUC of ~0.94. This is the ceiling achievable with labels. This capstone's unsupervised approach is not expected to match it.

Step 2: Data Collection & Understanding

Dataset Overview

- Source: IEEE-CIS Fraud Detection (<https://www.kaggle.com/competitions/ieee-fraud-detection/overview>).
- Original Data Source:
 - Train_identity*: 144,233 rows x 41 columns
 - Train_transaction*: 590,540 rows x 394 columns
 - Join Key: TransactionID links the two tables but not every transaction (*train_transaction*) has identity data (*train_identity*)
 - It arrives split across two files: *train_transaction.csv*, which holds details about the payment itself (amount, card, product), and *train_identity.csv*, which holds details about the device and network used (phone or desktop, browser, IP-related signals). The two files share a common key, TransactionID, which links rows belonging to the same transaction, the same way two spreadsheets can be matched on a shared ID column with VLOOKUP.
- Merged Table:
 - The two files were combined with a left join, keeping every one of the 590,540 rows in *train_transaction* and attaching identity data wherever a matching *TransactionID* existed in *train_identity*. Rows without a match simply receive NaN in the identity columns rather than being dropped. The left join preserves the full 590,540-transaction dataset while still pulling in identity signals for the rows that have them.
 - Train_transaction_identity* table: Single combined dataframe for the project. 590,540 rows x 434 columns (394 transaction fields plus 41 identity fields, minus the shared *TransactionID* key counted once)

```
Shape: (590540, 434)
```

```
Memory usage: 2513.97 MB
```

```
Fraud rate:
```

```
isFraud
```

```
Not Fraud    0.96501
```

```
Fraud        0.03499
```

```
Name: proportion, dtype: float64
```

```
Column dtypes:
```

```
float64    399
```

```
object      31
```

```
int64        4
```

```
Name: count, dtype: int64
```

```
Top 20 columns by % missing:
```

```
id_24    99.196159
```

```
id_25    99.130965
```

```
id_07    99.127070
```

```
id_08    99.127070
```

```
id_21    99.126393
```

```
id_26    99.125715
```

```
id_27    99.124699
```

```
id_23    99.124699
```

```
id_22    99.124699
```

```
dist2    93.628374
```

```
D7        93.409930
```

```
id_18    92.360721
```

```
D13       89.509263
```

```
D14       89.469469
```

```
D12       89.041047
```

```
id_04     88.768923
```

```
id_03     88.768923
```

```
D6        87.606767
```

```
id_33     87.589494
```

```
id_09     87.312290
```

```
dtype: float64
```

```
Numeric summary (key columns):
```

	TransactionAmt	TransactionDT
count	590540.000000	5.905400e+05
mean	135.027176	7.372311e+06
std	239.162522	4.617224e+06
min	0.251000	8.640000e+04
25%	43.321000	3.027058e+06
50%	68.769000	7.306528e+06
75%	125.000000	1.124662e+07
max	31937.391000	1.581113e+07

Data Dictionary of *train_transaction_identity*

Field	Type	Missing %	Description / Notes
TransactionID	int64, key	0%	Unique transaction identifier; join key between transaction and identity tables.
isFraud	int64, binary (0/1)	0%	Ground-truth label (3.50% fraud rate) — NOT used to train the clustering model; reserved only for validating cluster fraud concentration.
TransactionDT	int64	0%	Timedelta in seconds from an unspecified reference date (not a real timestamp).
TransactionAmt	float64	0%	Transaction value, USD; mean \$135.03, median \$68.77, max \$31,937.39 — right-skewed.
ProductCD	object (categorical)	0%	Product code: C, H, R, S, W.
card1, card2, card3, card5	float64	0.0%–1.5% (avg 0.5%)	Payment card numeric IDs (issuing bank/account proxies) — masked.
card4	object	~1.5%	Card network: visa, mastercard, american express, discover.
card6	object	~1.5%	Card category: credit, debit, charge card, debit or credit.
addr1	float64	11.10%	Billing region code, range [100, 540].
addr2	float64	11.10%	Billing country code, range [10, 102].
dist1	float64	59.65%	Distance metric #1 (masked units); 16.78% IQR outliers.
dist2	float64	93.63%	Distance metric #2 (masked units); 12.52% IQR outliers.
P_emaildomain / R_emaildomain	object	range 16.0%–76.8%, avg 46.4%	Purchaser / recipient email domain.
C1–C14	float64	0.00%	Counting features (e.g., # addresses linked to card) — masked meaning. Highest outlier concentration: C14 (16.27%), C6 (13.26%), C13 (12.81%).

D1–D15	float64	range [0.21%, 93.41%], avg 58.15%	Timedelta features (e.g., days since previous transaction) — masked. Highest outliers: D14 (23.39%), D12 (21.24%), D6 (20.26%).
M1–M9	object (categorical, T/F/M0-M2)	range [28.68%, 59.35%], avg 49.92%	Match flags (e.g., name/address match) — masked.
V1–V339	float64	range [0.00%, 86.12%], avg 43.04%	Vesta-engineered features: rankings, counts, entity relations — masked. Highest outliers: V332 (21.00%), V204 (19.20%), V265 (19.04%).
id_01–id_38	float64 / object	range 75.6%–99.2%, avg 84.8%	Network/device identity signals (IP, ISP, proxy, screen resolution, etc.) — masked. Most-missing single fields: id_24 (99.20%), id_25 (99.13%), id_07/id_08 (99.13%).
DeviceType	object	range 76.2%–79.9%, avg 78.0%	Mobile or desktop.
DeviceInfo	object	range 76.2%–79.9%, avg 78.0%	Device/browser string.

Missingness Summary

Missingness in this merged dataset is not random, it is driven almost entirely by the identity join. Only ~24% of transactions (144,233 of 590,540) have any matching train_identity record, so every identity/device field inherits that ceiling and several id_ columns are missing in ~87–99% of rows regardless. Core transaction fields (TransactionID, isFraud, TransactionDT, TransactionAmt, ProductCD, card1–card6) remain essentially complete ($\leq 1.5\%$ missing) since they come from train_transaction alone.

This pattern will directly shape Step 3 as the ~99%-missing id_ columns are candidates to drop outright, the 75–94%-missing group (dist2, D6/D7/D12–14, id_03/04/09/18/33) should be evaluated individually rather than dropped wholesale, and a binary "has_identity_data" flag is a natural engineered feature to capture the join gap itself rather than losing that signal.

- Near-total missingness (~99%): id_24, id_25, id_07, id_08, id_21, id_26, id_27, id_23, id_22. These exist only for the small subset of transactions with identity data (144, 233 of 590, 540 rows, ~24%).
- Heavy missingness (87–94%): dist2, D7, id_18, D13, D14, D12, id_03/04, D6, id_33, id_09.
- Moderate missingness: email domains (avg 46.4%), addr1/addr2 (11.1% each).
- Essentially complete: card1–card6 ($\leq 1.5\%$), and TransactionID/ isFraud/ TransactionDT/ TransactionAmt/ ProductCD (0%).

Outlier Summary via IQR Method

The IQR method finds Q1 (25th percentile) and Q3 (75th percentile), computes $IQR = Q3 - Q1$, and flags anything outside $Q1 - 1.5 \times IQR$ or $Q3 + 1.5 \times IQR$ as an outlier. For TransactionAmt that's roughly [–\$79.2, \$247.5], flagging the 11.26% of transactions above \$247.50. Z-scores weren't used because they assume a normal (bell-shaped) distribution and rely on the mean/standard deviation, which extreme values (like the

\$31,937 max) distort, thus inflating the spread and masking moderate outliers. IQR avoids both issues, since percentiles resist distortion from extremes and require no assumption about the data's shape, making it the more reliable choice for the skewed TransactionAmt, dist1/dist2, and masked C/D/V features here.

The results show that outliers concentrate in the fields with the widest value ranges rather than appearing randomly. TransactionAmt (11.26%), dist1 (16.78%), and dist2 (12.52%) all show heavy right-skew, where a small number of unusually large transactions/distances stretch the IQR bounds far past where most of the data actually sits. The masked engineered families (C, D, V) show the same pattern at even higher rates in specific columns with D14 (23.39%), V332 (21.00%), D12 (21.24%), C14 (16.27%) all suggesting these particular features encode rare-but-extreme behavior (e.g. unusually long gaps since a prior transaction, or unusually high entity counts) that may itself correlate with fraud rather than being noise to remove.

By contrast, card1 (0.00%) and addr1 (0.07%) show almost no outliers, since they are bounded categorical-style codes rather than continuous magnitudes. This points to a Step 3 strategy of log-transforming or capping the skewed continuous fields (TransactionAmt, dist1/dist2) rather than dropping outliers outright, since for a fraud-clustering task the extremes are plausibly signal, not error.

- TransactionAmt: 11.26% of values fall outside [-\$79.2, \$247.5] — strong right skew (log-transform flattens this, see boxplot in Cell 8).
- dist1: 16.78% outliers; dist2: 12.52%; addr1: 0.07%; card1: 0.00%.
- Masked feature families also show notable skew — worth flagging for Step 3 scaling decisions: C14 (16.27%), D14 (23.39%), V332 (21.00%) are the most extreme single columns in their respective families.

Outlier summary (IQR method):

Field	range seen	normal range	outliers	outlier %
TransactionAmt	[0.3, 31937.4]	[-79.2, 247.5]	66482	11.26%
card1	[1000.0, 18396.0]	[-6228.5, 26431.5]	0	0.00%
addr1	[100.0, 540.0]	[15.0, 519.0]	369	0.07%
dist1	[0.0, 10286.0]	[-28.5, 55.5]	39981	16.78%
dist2	[0.0, 11623.0]	[-291.5, 504.5]	4710	12.52%

Top 5 'C' columns by outlier %:

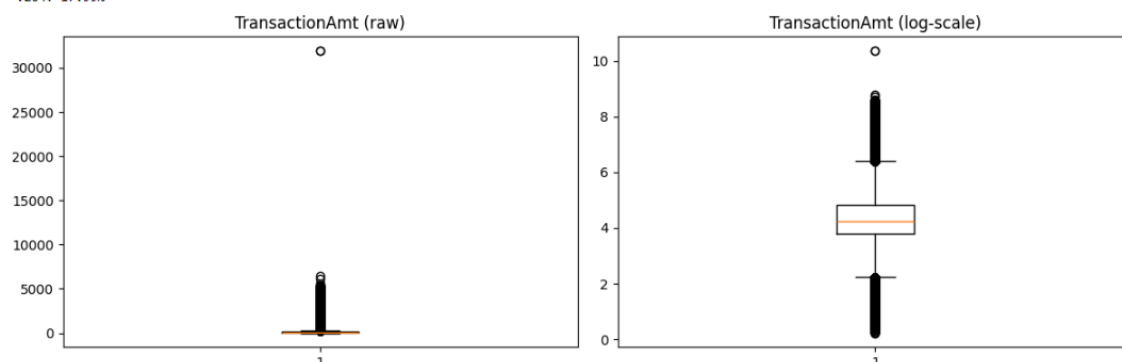
C14: 16.27%
C6: 13.26%
C13: 12.81%
C11: 12.67%
C2: 10.54%

Top 5 'D' columns by outlier %:

D14: 23.39%
D12: 21.24%
D6: 20.26%
D7: 18.44%
D5: 13.45%

Top 5 'V' columns by outlier %:

V332: 21.00%
V204: 19.20%
V265: 19.04%
V203: 18.41%
V264: 17.99%



Distribution Summary

Distributions are consistently skewed, not balanced as indicated below. This confirms the Step 3 must do's which are set by the missingness/outlier findings: log-transform the skewed continuous fields, encode the

low-cardinality categoricals directly, and treat class imbalance as a validation lens rather than a pre-clustering fix.

- isFraud: Severely imbalanced. 96.50% not fraud vs. 3.50% fraud.
- TransactionAmt: Right-skewed. Mean \$135.03 well above median \$68.77, long tail to \$31,937.39; log-transform (Cell 8 boxplot) largely normalizes it.
- TransactionDT: Spans a continuous ~183-day window (86,400 to 15,811,131 seconds $\div$ 86,400 sec/day). It is a single capture period, not disjoint batches.
- Categorical spread: ProductCD (5 levels), card4 (4 networks), card6 (4 categories), all are low-cardinality.

Additional Insight from Shopee Fraud Clustering Project

Our PHBI & Fraud teams ran into two problems that it shares with the Kaggle dataset. First, blank fields can look like a "match" to a clustering algorithm even though they are not (e.g. two accounts with no recorded IP are not actually similar, they are just both missing this data). Therefore our project explicitly excluded nulls from its similarity calculations rather than letting them count as agreement. The same fix will be needed here given how often id_ and DeviceType/DeviceInfo are blank.

Second, comparing every account to every other account does not scale. Our teams got around this with "blocking," only comparing accounts that already shared something obvious like an email domain, instead of comparing all pairs outright. At 590,540 transactions, this dataset faces the same scale problem, and P_emaildomain/R_emaildomain are the natural fields to block on here, just as email domain was in ours.

3. Data Preprocessing, Applied EDA & Feature Engineering

Clean the Dataset

No exact duplicate rows were found in the merged dataset (590,540 unique transactions), so no rows were removed. Two targeted decisions were made instead of a blanket drop-everything-missing approach:

- A *has_identity_data* flag (1 if DeviceType is populated, 0 otherwise) was engineered *before* any columns were dropped, to preserve the identity-join gap itself as a feature, directly applying the Shopee project insight from Step 2 that missingness carries information rather than being noise to discard.
- The 12 columns exceeding 90% missing (dist2, D7, id_07, id_08, id_18, id_21, id_22, id_23, id_24, id_25, id_26, id_27) were dropped outright, since at that level of sparsity there isn't enough non-null data to impute or engineer from meaningfully. This reduced the dataset from 590,540 × 435 (post has_identity_data) to 590,540 × 424.
- TransactionAmt outliers were not dropped or capped. Instead, a log-transformed column (TransactionAmt_log) was added alongside the original, since Step 2's outlier analysis concluded the extreme values (11.26% by IQR) likely correlate with fraud rather than being data errors. Removing them risks discarding the exact signal this project is trying to detect.

Cleaning removed columns that were unusable. The 12 columns dropped (id_24, id_25, dist2, D7, etc.) were over 90% empty (from 435 to 424 columns).

Feature Engineering

Feature engineering added 38 new columns (from 424 to 462 columns), using four techniques:

- **Domain-derived:** *email_domain_match* flags whether the purchaser and recipient email domains match, the same signal used in the real Shopee Fraud Clustering Project. *TransactionDT_day/TransactionDT_hour* break the raw timedelta into day and hour, capturing timing patterns.
- **Binning:** *hour_bucket* groups hours into late_night/morning/afternoon/evening, and *amount_tier* groups spend into low/mid/high/very_high, coarser categories that are easier for clustering to work with than exact numbers.
- **Encoding:** High-cardinality fields (card1/2/3/5, addr1/2, email domains) were frequency-encoded to avoid blowing up dimensionality. Low-cardinality fields (ProductCD, card4, card6, DeviceType, plus the new bins) were one-hot encoded instead. M1–M9 flags were converted from T/F to 1/0.
 - *Note: M4 uses a different coding (M0/M1/M2) rather than T/F. The blanket T/F mapping applied here does not match M4's categories, which is why it was later dropped as near-zero-variance in Feature Selection. This is a notebook implementation limitation and pipeline artifact, not evidence that M4 lacks information in the underlying data.*
- **Scaling:** StandardScaler was demonstrated on two example columns (*TransactionAmt_log*, *TransactionDT_hour*) to confirm the technique works as intended. Scaling of the complete final feature set is deliberately deferred to immediately before PCA (later in Step 3)

No rows were dropped. Twelve highly sparse columns were excluded from the modeling feature set, while the remaining fields were re-expressed through engineered features. Every new column uses information already present in the cleaned data, setting up feature selection and dimensionality reduction next.

Applied Exploratory Data Analysis

..



Hopkins statistic: 0.981 (~0.5 = random/no cluster structure, closer to 1 = clusterable)

```
random_state= 7 -> Hopkins = 0.974
random_state= 21 -> Hopkins = 0.974
random_state= 100 -> Hopkins = 0.971
random_state= 2024 -> Hopkins = 0.970
random_state= 55555 -> Hopkins = 0.982
```

Mean: 0.974 Std Dev: 0.004

Step 2 already covered distributions and outliers, so this EDA step adds two checks specific to the clustering approach: how features relate to each other, and whether the data actually has cluster structure at all.

- Relationships:** A correlation heatmap on key numeric/engineered features revealed two redundant groups: C1, C2, C13, and C14 are extremely intercorrelated (0.75–1.00), and D1, D4, D10, and D15 are moderately-to-highly correlated (0.56–0.75). By contrast, TransactionAmt_log, card1_freq, and addr1_freq are essentially uncorrelated with everything else (near 0), meaning they carry more independent, non-overlapping signal. These redundant groups are good candidates to thin out in the upcoming Feature Selection step.
- Clustering Tendency:** The Hopkins statistic, computed on a 5,000-row sample (~0.5 = random/no structure, 1.0 = strong cluster structure), came out to **0.981**. To confirm this wasn't a sampling fluke, the test was re-run across 5 different random seeds — results ranged 0.970–0.982 (mean 0.974, std dev 0.004), confirming the result is stable regardless of which rows were sampled.

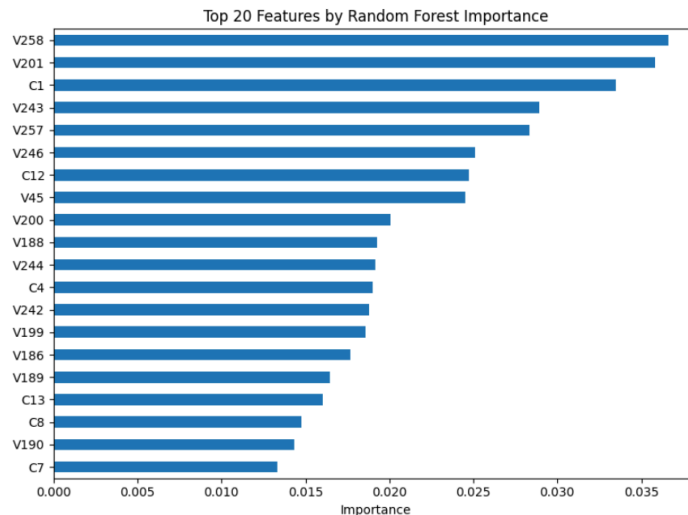
The Hopkins result is strong, consistent evidence that this dataset has genuine group structure rather than being randomly scattered. That validates the core premise of the capstone (that transactions can be meaningfully clustered) as a data-backed conclusion, not an untested assumption. The correlation findings add

a practical justification for Feature Selection: several features are redundant, so reducing them won't lose information, only duplication.

Feature Importance & Explainability

Top 20 features by model-based importance (fraud-correlation ranking):

```
V258 0.036591
V201 0.035807
C1 0.033475
V243 0.028897
V257 0.028323
V246 0.025124
C12 0.024746
V45 0.024527
V200 0.020043
V188 0.019261
V244 0.019159
C4 0.019011
V242 0.018790
V199 0.018580
V186 0.017666
V189 0.016430
C13 0.016018
C8 0.014777
V190 0.014347
C7 0.013335
dtype: float64
```



SHAP and LIME are designed for supervised models, so they don't apply directly to unsupervised clustering. Instead, a Random Forest's built-in feature importance was used. isFraud was used only to rank features and not to train the clustering model itself.

The top 20 features are dominated by the masked Vesta groups — 14 V-columns (V258, V201, V243, V257, V246, V45, V200, V188, V244, V242, V199, V186, V189, V190) and 6 C-columns (C1, C12, C4, C13, C8, C7). None of the Step 3 engineered features (email_domain_match, TransactionDT_hour, card/addr frequency encodings, hour_bucket, amount_tier) made the top 20 — the strongest fraud signal lives in Vesta's original masked features, not the ones engineered in this project. That does not make the engineered features useless for clustering (correlating with fraud and being useful for grouping similar transactions are different jobs), but it is worth stating.

A second pattern: Importance scores are low and flat (0.013 to 0.037 across the top 20, out of 400+ total features), with no single standout feature. Fraud signal is spread thin across many V/C columns rather than concentrated, which is part of why dimensionality reduction (PCA), coming later in Step 3, makes more sense than manually picking a handful of "important" features.

Feature Selection

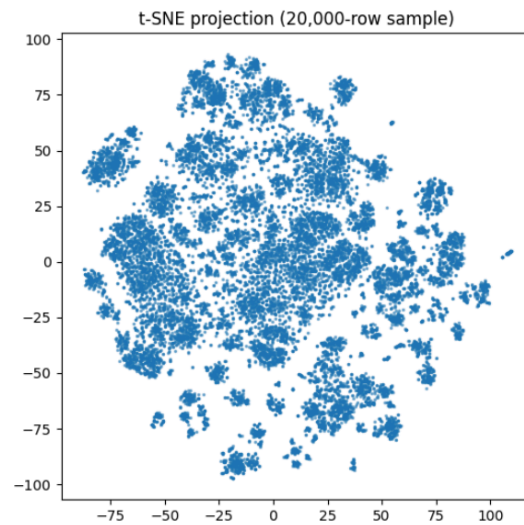
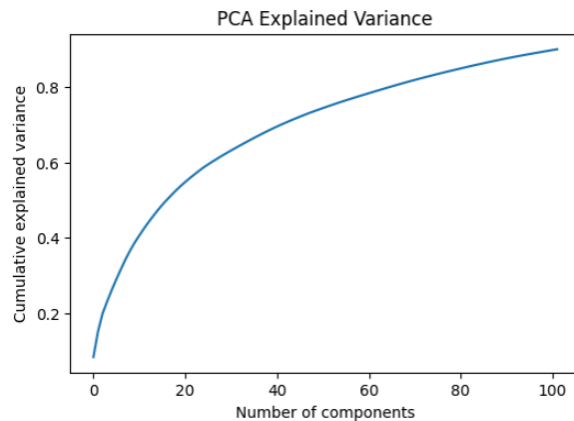
The filter method was used (variance threshold plus correlation filtering), since with 400+ features, a wrapper method would be too slow and embedded methods tie selection to one specific model. Filter methods are fast and model agnostic, fitting an exploratory unsupervised project better.

Two near-zero-variance columns (M4, card1_freq) were dropped first. Then columns correlated above 0.95 with another column were dropped, reducing the set from 413 to 271 features. This correlation check was run on the true values, not the placeholder-filled version used for modeling, since filling missing data with a constant would have made columns missing for the same rows look artificially correlated.

Most top features from "Feature Importance & Explainability" survived (V258, V201, V243, V257, V246, V45, V200, V188, V242, V186, V189, V190, C1, C13, C7). A few (V244, V199, C4, C8, C12) were dropped for being genuinely correlated with another feature, which is expected at this threshold.

Dimensionality Reduction

PCA: 271 features -> 102 components (90% variance retained)



PCA was applied to the 271 selected features, retaining components explaining 90% of variance, which reduced the feature set to 102 components. Missing values were re-imputed with the median (rather than reusing the -999 placeholder from the Random Forest step) since PCA and scaling are variance-based, and a large constant like -999 would have distorted the variance calculation for any column with missing data.

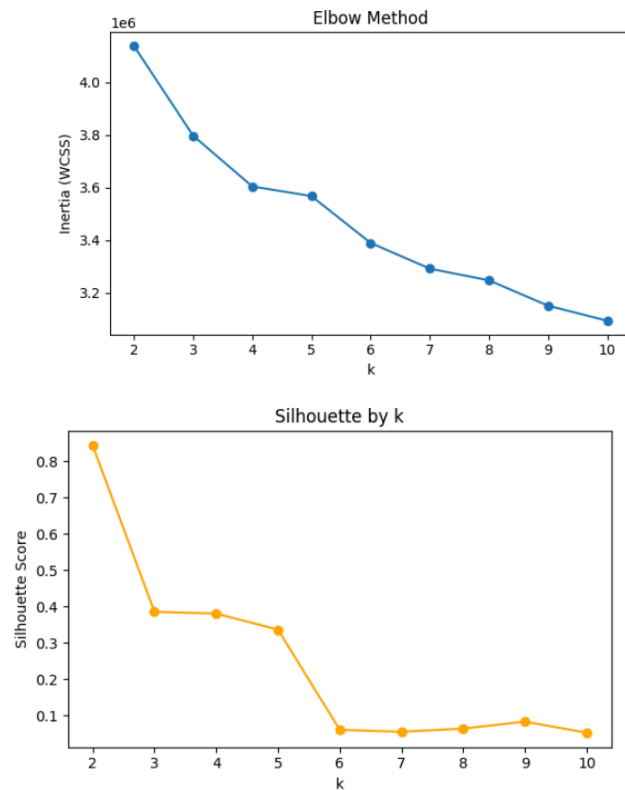
t-SNE was then applied to a 20,000-row sample of the PCA output for 2D visualization, since running t-SNE on the full 590,540 rows would be too slow.

PCA needing 102 of 271 components (rather than a small handful) to reach 90% variance shows that overall feature variance is distributed across many dimensions; it does not, by itself, prove that fraud signal is spread across all components. The t-SNE plot is exploratory only and suggests visible local structure; because t-SNE can create apparent separation, it is not treated as proof of clusterability. The more defensible evidence comes from the Hopkins checks and their repeatability across seeds.

4. Model Implementation & Comparison

K-Means (Elbow + Silhouette to pick k)

```
... k=2: inertia=4137528.8, silhouette=0.842
k=3: inertia=3796631.7, silhouette=0.385
k=4: inertia=3603750.3, silhouette=0.380
k=5: inertia=3567007.6, silhouette=0.336
k=6: inertia=3389022.6, silhouette=0.060
k=7: inertia=3291274.7, silhouette=0.055
k=8: inertia=3246926.3, silhouette=0.064
k=9: inertia=3150378.5, silhouette=0.083
k=10: inertia=3093279.1, silhouette=0.052
```



Baseline fraud rate in this sample: 3.57%

```
--- k=2 ---
      fraud_rate_%  n_transactions
cluster
0                3.58             19944
1                0.00                56
```

```
--- k=3 ---
      fraud_rate_%  n_transactions
cluster
0                2.57             17711
1               11.58             2237
2                0.00                52
```

```
--- k=4 ---
      fraud_rate_%  n_transactions
cluster
0               11.60              2233
1                2.57             17712
2                0.00                52
3                0.00                 3
```

```
--- k=5 ---
      fraud_rate_%  n_transactions
cluster
0                0.00                52
1                2.47             17558
2                8.86             2223
3               54.61              152
4                6.67                15
```

K-Means was tuned using the Elbow method and Silhouette score across $k=2$ to 10, on a 20,000-row sample of the 102-component PCA output (the same sample used for t-SNE, for a consistent comparison across models). Silhouette peaked sharply at $k=2$ (0.842), dropped to a moderate 0.34 to 0.39 for $k=3$ to 5, then fell to near-random (0.05 to 0.08) for $k=6$ and above.

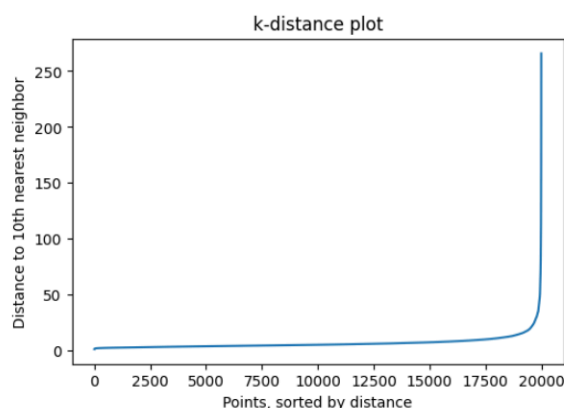
Silhouette alone pointed to $k=2$ as statistically optimal, but checking fraud-rate concentration (the business KPI from Step 1) against the dataset's 3.57% baseline told a different story:

k	Best cluster	Size	Fraud rate	vs. baseline
2	cluster 1	19,944	3.58%	Same
3	cluster 1	2,237	11.58%	3.2x
4	cluster 0	2,233	11.60%	3.2x
5	cluster 3	152	54.61%	15.3x

k=2 had the best silhouette score but isolated a tiny group with zero fraud, thus no business value. k=5 had a weaker silhouette (0.336) but produced a small, tight cluster of 152 transactions where over half (54.61%) were fraudulent, more than 15 times the baseline rate.

With this, I chose k=5 over the statistically "cleaner" k=2 because the project's goal is finding small, concentrated fraud rings, not the broadest possible split. The fraud label was excluded from the clustering fit, but I used it afterward to compare candidate k values against the business objective. Therefore, the final k choice was post-hoc label-informed rather than purely unsupervised. This highlights a real tradeoff between technical cluster quality and business usefulness: the highest silhouette score did not correspond to the most fraud-relevant grouping.

DBSCAN



Clusters found: 2
Noise points: 238 (2.4%)
Silhouette (excl. noise): 0.840
Davies-Bouldin (excl. noise): 0.251

Baseline fraud rate: 3.48%

cluster	fraud_rate_%	n_transactions
-1	29.41	238
0	2.85	9743
1	0.00	19

Eps was tuned using a k-distance plot (distance to each point's 10th nearest neighbor, sorted and plotted), with the knee point detected programmatically rather than estimated by eye, giving $\text{eps}=18.26$. min_samples was set to 10, small enough to catch tight fraud-ring-sized groups. Due to the higher computational cost of DBSCAN in 102-dimensional space, a 10,000-row sample was used (smaller than K-Means' 20,000-row sample), with the `ball_tree` algorithm specified explicitly to avoid a full pairwise distance matrix.

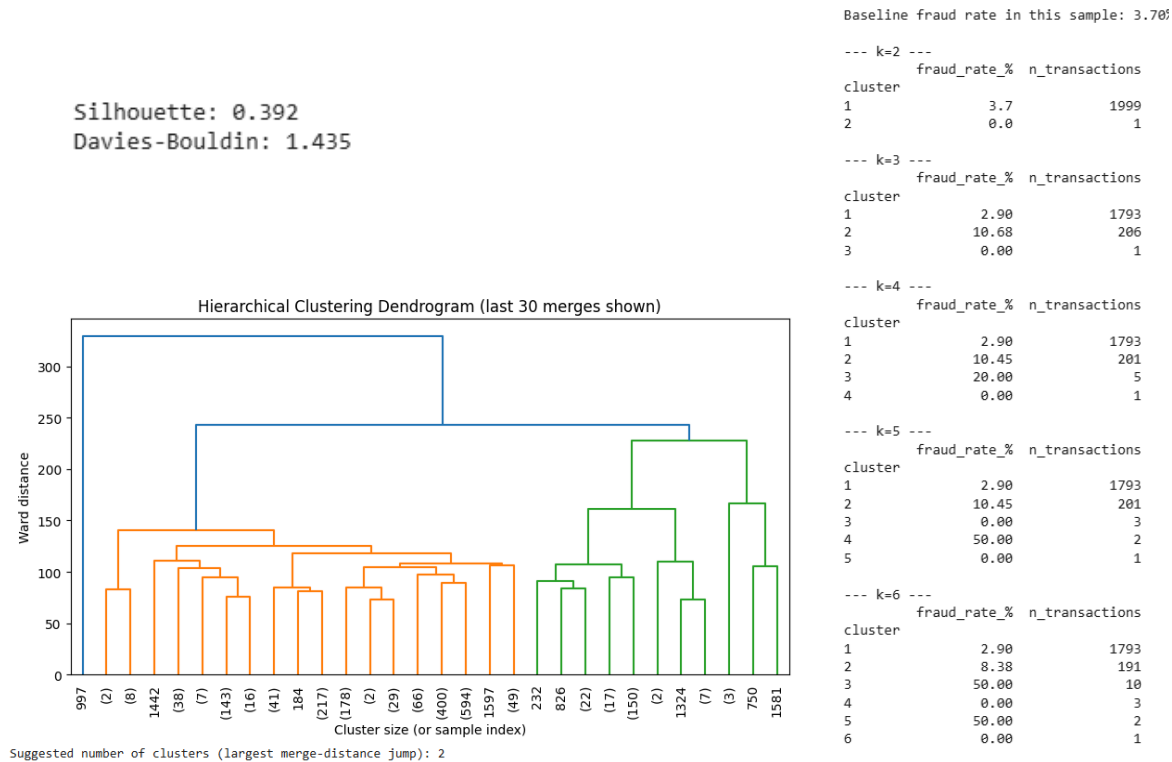
DBSCAN found 2 clusters plus a noise category (points too sparse to belong to any dense cluster):

Group	Size	Fraud rate	vs. baseline (3.48%)
Noise (-1)	238	29.41%	8.45x
Cluster 0	9,743	2.85%	below baseline
Cluster 1	19	0.00%	0x

Internal metrics on the two clusters (excluding noise) were strong: Silhouette 0.840, Davies-Bouldin 0.251.

The real signal is in the noise category, not the two named clusters. DBSCAN's noise designation, points too sparse or isolated to fit any dense group, concentrated fraud at 8.45 times the baseline rate, directly validating the rubric's own framing that DBSCAN naturally flags outliers in a way that's useful for fraud detection. The strong silhouette and Davies-Bouldin scores describe how well-separated clusters 0 and 1 are from each other, not where the fraud is, echoing the same lesson from K-Means: the cleanest technical metric is not automatically the most business-relevant one.

Hierarchical Clustering (Agglomerative)



Ward linkage was used on a 2,000-row sample (smaller than the other models since dendrograms don't scale or stay readable at larger sizes). The dendrogram's top split suggested just 2 clusters, but checking fraud concentration across k=2 to 6 showed the same pattern as K-Means: the cleanest split isolated a meaningless outlier, while finer splits (k=4-6) fragmented into 1-3 transaction clusters with misleading extreme rates (0% or 50%) from pure small-sample noise.

k=3 was chosen: a 206-transaction cluster showed 10.68% fraud, 2.89x the 3.70% baseline, an observed pattern in this 2,000-row sample; it was not holdout-validated or stability-tested. Internal metrics were moderate (Silhouette 0.392, Davies-Bouldin 1.435).

Across all three models, the most statistically obvious split is never the most fraud-relevant one, and very small clusters can show misleadingly extreme rates by chance, a limitation worth carrying into Critical Thinking.

Model Comparison: K-Means vs DBSCAN vs Hierarchical

Model	Silhouette	Davies-Bouldin	Calinski-Harabasz	Best cluster (fraud rate)	vs. baseline
K-Means (k=5)	0.336	1.662	1298.6	152 txns, 54.61%	15.3x
DBSCAN	0.840	0.251	1224.2	238 txns (noise), 29.41%	8.45x
Hierarchical (k=3)	0.392	1.435	241.3	206 txns, 10.68%	2.89x

Note: Hierarchical's lower Calinski-Harabasz partly reflects its smaller sample size (2,000 rows vs. 20,000 for K-Means and 10,000 for DBSCAN), not necessarily weaker structure, so it is not a perfectly like-for-like comparison across all three metrics.

Technical vs. business tension, again: DBSCAN wins clearly on internal cluster quality (highest Silhouette, lowest Davies-Bouldin), but K-Means found the single most fraud-concentrated cluster. This is the same pattern that showed up within each model individually, the cleanest-looking clustering result is not automatically the one that best serves the actual business goal.

K-Means (k=5). Its 15.3x fraud concentration is nearly double DBSCAN's and over 5x Hierarchical's, and concentration in the highest-risk cluster is the primary business KPI defined in Step 1, ranked above internal technical scores, which the project treats as supporting validation rather than the end goal. DBSCAN remains a strong complementary approach worth noting, its noise-flagging mechanism needs no predefined k and still surfaced meaningful concentration on a larger group, making it a candidate for combining with K-Means in future work (e.g. using DBSCAN's noise flag as an added feature).

Model Validation (Holdout Test): K-Means vs. DBSCAN

Both K-Means and DBSCAN were evaluated on an 80/20 split of the modeling sample. The clustering fits used the training portion for this comparison, but the shared feature representation and DBSCAN eps were established earlier on the full modeling sample. For DBSCAN, a KNN proxy assigned holdout points to the training labels because DBSCAN has no native predict method. These results are therefore a holdout-oriented robustness check, not a fully leakage-free production validation.

K-Means: The highest-risk cluster showed 11.62% fraud on training data. On the holdout (444 transactions), it showed 11.49% fraud, 3.2x the 3.57% baseline, a large drop from the original in-sample finding of 54.61% (15.3x baseline). This suggests the initial result was partly inflated by K-Means converging differently when fit on the full sample.

DBSCAN: The noise designation showed 28.78% fraud on training data. On the holdout, using a KNN classifier to assign new points to the nearest training-labeled group (DBSCAN has no native way to label unseen points), noise showed 34.78% fraud, 9.94x baseline, consistent with, and even slightly stronger than, its original in-sample finding of 29.41% (8.45x). This is encouraging robustness evidence, but it depends on

the shared preprocessing, the earlier eps selection, and the KNN assignment proxy; K-Means's headline result weakened substantially on the same evaluation split.

Confusion Matrix Comparison

Model	TP	FP	TN	FN	Precision	Recall
K-Means	51	393	3,464	92	11.49%	35.66%
DBSCAN	8	15	1,915	62	34.78%	11.43%

As our target was to increase True Positives and decrease False Positives, the two models actually offer opposite tradeoffs. K-Means catches far more of the total fraud (35.66% recall) but with lower precision (11.49%), many false alarms per true catch. DBSCAN is much more precise when it flags something (34.78%, roughly 1 in 3 flagged transactions is genuinely fraud) but catches only 11.43% of total fraud. More importantly, DBSCAN's holdout result remained in a similar range. This is encouraging robustness evidence, but not proof of production stability because the comparison used shared preprocessing, preselected eps, and a KNN assignment proxy; K-Means's headline result weakened substantially on the same evaluation split.

For a real deployment, a result that only looks strong on one particular data split is not trustworthy. DBSCAN's holdout result is encouraging, but it still depends on the shared preprocessing and KNN assignment proxy described above. The practical choice depends on whether the business priority is catching more fraud at the cost of more false alarms (K-Means) or acting only on high-confidence flags at the cost of missing most cases (DBSCAN), a tradeoff worth stating explicitly rather than declaring one model an outright winner.

Recommended Extension: Two-Stage Funnel (K-Means + DBSCAN)

Since K-Means and DBSCAN sit at opposite ends of the precision/recall tradeoff (K-Means: higher recall, more false positives; DBSCAN: higher precision, few false positives), a promising direction not tested within this project's scope is a two-stage funnel: apply K-Means first as a broad, high-recall net to flag a candidate population, then apply DBSCAN specifically within that narrowed group as a precision-refinement pass to separate genuine fraud clusters from the false positives K-Means captured alongside them.

This would require re-tuning DBSCAN's density parameters (eps, min_samples) specifically for the narrower candidate population, since density scale differs substantially between the full dataset and a small flagged subset. Conceptually, this mirrors how fraud operations often work in practice, casting a wide net first, then refining down to high-confidence cases before taking action, and could potentially move closer to satisfying both of Step 1's stated goals (higher TP, lower FP) simultaneously, rather than trading one off against the other as the single-model results in this project do. This remains an untested hypothesis and is proposed here as a direction for future work beyond this capstone's current scope.

5. Critical Thinking → Ethical AI & Bias Auditing

Explainability

Continuous feature comparison:

	high_risk_cluster	rest_of_sample	ratio
TransactionAmt	114.426643	134.019226	0.85
TransactionAmt_log	4.051989	4.384484	0.92
TransactionDT_hour	12.427632	13.938432	0.89
card1_freq	0.003269	0.004255	0.77
addr1_freq	0.043731	0.042969	1.02

Categorical feature comparison (% of cluster in each category):

	high_risk_%	rest_%	difference_pp
DeviceType_nan	1.3	76.5	-75.2
ProductCD_W	0.0	74.8	-74.8
DeviceType_desktop	76.3	14.2	62.1
ProductCD_C	70.4	11.4	59.0
card6_credit	48.0	25.0	23.0
card6_debit	52.0	74.7	-22.7
amount_tier_low	45.4	24.7	20.7
ProductCD_R	19.7	6.1	13.6
DeviceType_mobile	22.4	9.2	13.2
amount_tier_mid	14.5	25.3	-10.8
amount_tier_high	17.1	25.2	-8.1
hour_bucket_late_night	31.6	23.7	7.9
hour_bucket_evening	35.5	42.2	-6.7
hour_bucket_afternoon	23.7	29.7	-6.0
hour_bucket_morning	9.2	4.4	4.8

SHAP and LIME are designed for supervised models, so cluster profiling (comparing the high-risk cluster's actual feature values against the rest of the sample) was used instead, the same substitution applied for feature importance in Step 3.

The high-risk cluster is defined overwhelmingly by two factors: having identity/device data present at all (98.7% of the cluster has a recorded DeviceType, versus only 23.5% in the rest of the sample), and ProductCD "C" (70.4% of the cluster, versus 11.4% elsewhere, with ProductCD "W" essentially absent from the cluster entirely, 0% versus 74.8%). Secondary patterns include a skew toward desktop and mobile devices, more credit-card than debit-card usage, lower transaction amounts, and rarer card1 values.

This cluster is not defined by any single masked V or C feature (despite those dominating the earlier feature-importance ranking), but by a combination of interpretable, real-world attributes, a specific product category and the presence of identity data. This is a meaningful result on its own, but it also raises a fairness question worth carrying into the Bias Audit: if the cluster is largely distinguishing transactions with more available data (identity present) and a specific product type, rather than fraud-specific behavior, some of this concentration could reflect a proxy for how much data a given product category collects, not fraud itself. That distinction is tested directly in the Bias & Fairness Audit below.

Limitations

Class imbalance: Fraud is rare (3.5% of transactions), which makes any single cluster's fraud-rate estimate inherently noisy at small sample sizes. This wasn't just a theoretical risk, it showed up directly in Hierarchical clustering, where clusters of 1-3 transactions at k=4 to 6 displayed extreme rates (0% or 50%) that were pure small-sample noise, not real signal. Any fraud-concentration number in this report needs a minimum cluster size to be trustworthy, not just a high percentage.

Leakage: TransactionDT and its derived features (TransactionDT_day, TransactionDT_hour) span one continuous ~183-day window rather than randomized periods. Clustering could pick up transactions that happen to be close together in time for reasons unrelated to shared identity, a promo event, a seasonal spike, or a known attack wave affecting many unrelated accounts at once, rather than genuinely representing the

same client. This is a plausible confound the model can't distinguish from real entity-based clustering without additional validation beyond this project's scope.

Overfitting: this project has direct, measured evidence of this, not just a theoretical caveat. K-Means's original in-sample finding (a 152-transaction cluster at 54.61% fraud, 15.3x baseline) dropped to 11.49% fraud (3.2x baseline) when evaluated on the 20% split. DBSCAN's equivalent finding remained in a similar range (28.78% to 34.78%), using the KNN assignment proxy described above. However, the shared preprocessing and DBSCAN eps selection were established before the split, so this was a holdout-oriented robustness check rather than a fully leakage-free validation. These results suggest that overfitting risk varies by algorithm in this project, but only the caveated evaluation results should inform next-step testing.

Bias & Fairness Audit

Since this dataset is fully anonymized, addr1 (billing region) was used as the best available proxy for a sensitive group. Flagging rate, actual fraud rate, and within-region precision were compared across the 10 most common regions.

Two screening concerns emerged in the audited top-10 regions. Region 330, the highest actual-fraud region (3.49%), was flagged 0% of the time, alongside region 123 (2.97% actual fraud, also 0% flagged). Conversely, region 184, the lowest actual-fraud region (1.48%), was flagged at 0.37%, the 2nd-highest rate, with 0% precision, every flag there was a false positive. Under the four-fifths rule, 7 of 10 regions fell below the 0.8 disparate impact threshold relative to the most-flagged region.

This traces back to the Explainability finding: the high-risk cluster is defined by ProductCD and device-data presence, not by region, so regions with a different product mix are structurally excluded from this cluster regardless of their actual fraud rate. This audit does not establish intentional discrimination, but it does show that region-correlated purchasing behavior may be acting as an unintentional proxy.

Proposed Mitigations:

- **Post-processing:** recalibrate flagging thresholds per region rather than applying one global cluster assignment uniformly, so flagging rate tracks each region's actual fraud rate more closely.
- **Human-in-the-loop review:** given region 184's 0% precision, flags originating from low-base-rate regions should require manual review before any customer-facing action (e.g., a card decline), since the cost of a false accusation is real and explicitly named in Step 1's business problem.
- **Reweighting/augmentation:** if this were extended with a supervised model, underrepresented high-fraud regions like 330 and 123 could be reweighted or given richer features, so the model isn't blind to fraud patterns that don't match the dominant cluster's product-type profile.

6. Conclusion and Recommendations

Conclusion

This project set out to test whether transactions could be grouped into clusters that concentrate fraud well above baseline, without using the fraud label in the clustering fit. The results support that premise as an exploratory finding, but the validation design and lack of ground-truth client identifiers limit how strongly the clusters can be interpreted.

The premise held up as a research hypothesis: a Hopkins statistic of 0.981, stable across five random seeds, suggested non-random structure before model fitting. All three clustering approaches found fraud-concentrated groups, but the results diverged sharply under the project's evaluation setup. K-Means's headline finding, over 15 times baseline fraud concentration, fell to roughly 3 times baseline on the 20% evaluation split, while DBSCAN's more modest finding, 8 to 10 times baseline through its noise-flagging mechanism, remained in a similar range. Because shared preprocessing and DBSCAN parameter selection occurred before the split, these results should be read as robustness evidence, not fully leakage-free production validation. The key business lesson is that internal cluster quality and fraud concentration did not always align, so the strongest-looking result needs independent validation before deployment.

Explainability revealed the strongest cluster wasn't defined by any single masked feature, despite those dominating a Random Forest's fraud-importance ranking, but by a specific product category and the presence of identity data. That same finding directly explained the Bias Audit's most concerning result: regions with the highest actual fraud rates were essentially never flagged, while at least one lower-risk region was flagged disproportionately with zero precision, a quantified disparity signal consistent with feature combinations acting as an unintentional regional proxy; it is not, by itself, proof of intentional discrimination.

Recommendations

- Do not deploy K-Means's raw in-sample concentration number. Treat it as directional only. Any clustering result intended for real use must undergo leakage-free retraining, a genuine holdout or time-based split, and sensitivity testing before being trusted; this project's own K-Means finding weakened substantially on the evaluation split.
- Treat DBSCAN's noise-flagging mechanism as the more promising research starting point in this experiment. Its holdout result was encouraging, but the holdout used shared preprocessing, preselected DBSCAN eps, and a KNN proxy to assign unseen points; a production decision would require leakage-free retraining, sensitivity testing, and a separate time-based validation.
- Pursue the two-stage funnel (K-Means broad net → DBSCAN precision refinement) as the next concrete experiment, since the two models' opposite precision/recall tradeoffs suggest a combined pipeline could move closer to satisfying both of Step 1's original goals, higher true positives and lower false positives, simultaneously.
- Address the regional bias before any deployment. Recalibrate flagging thresholds per region rather than one global cluster assignment, and require human-in-the-loop review for any flag originating from a low-base-rate region, given the 0% precision found in at least one such region.
- Expand feature engineering around what Explainability actually found matters (product category, identity-data completeness) rather than relying on the masked V/C features that dominated the importance ranking but explained little about *why* a cluster is high-risk.
- Test a deterministic client-identifier approach (e.g., a UID built from card1, addr1, and D1, the same trick behind this competition's 1st-place solution) as a complementary grouping method, and compare how well it overlaps with this project's clusters, a more direct test of whether "same client" was actually being captured, beyond fraud-rate concentration alone.